

Horizon: Dependency-Aware Autonomous Infrastructure Recovery

Architecture, Prior-Art Analysis and Patentability Assessment

A Technical Research Paper in Support of an Invention Disclosure

Sarthak Tulsidas Patil, Karan Verma, Sneha Sharma, Sneha Patidar

Independent Research / Hackathon Prototype, India

October 2026

Abstract

When distributed infrastructure breaks, restarting a single component rarely fixes anything. Failures travel along dependency chains, and bringing an application back before its database is healthy simply restarts the failure loop and stretches the mean time to recovery (MTTR). This paper presents Horizon, an offline-capable recovery platform that understands those dependencies. It keeps a directed graph of infrastructure nodes, works out the blast radius as soon as a failure is detected, builds a recovery sequence that respects the dependency order, runs low-risk steps automatically, holds high-risk steps for human or cryptographic approval, and records everything it does in an audit trail.

We describe Horizon as it is actually built today, not as it was first planned: a TypeScript/React monorepo with a pure in-browser recovery engine, Firebase real-time state and MST blockchain approval signatures. We then compare it with academic self-healing research, commercial AIOps and orchestration tools (StackStorm, Rundeck, Kubernetes controllers, Datadog, ServiceNow and Rubrik Autonomous Business Recovery) and issued patents, above all US12556441B1 (Bank of America, 2026) on automated dependency mapping for ordered remediation.

Our conclusion is measured. Each mechanism on its own (dependency graphs, topological sorting, blast-radius analysis, human approval gates, audit logs) is already known. What may still be distinguishable is the combination: a closed loop that generates and executes a dependency-constrained recovery sequence in real time, on live topology rather than a static runbook, with mandatory approval for high-risk steps and a cryptographic audit record. We say plainly that US12556441B1 and related prior art put broad claims at serious novelty risk. The paper closes with an evaluation plan, a security analysis, the limits of the current prototype and a practical IPR roadmap for an Indian team considering a provisional filing.

Keywords: *autonomous recovery, dependency graph, blast radius, AIOps, self-healing infrastructure, human-in-the-loop, topological recovery sequencing, prior art, patentability*

1. Introduction

1.1 Background

Modern enterprise infrastructure is, in effect, a directed graph of dependencies. Databases sit under APIs, APIs sit under frontends, caches and identity services sit in between, and a network or control-plane failure can cut off a whole subgraph at once. The usual recovery story, “find the failed node and restart it”, ignores the order in which things must come back. Restart an application while its data store is still down and you get crash loops, alert storms and a longer outage.

Day to day, recovery still depends on people coordinating in war rooms. Static runbooks describe yesterday’s

topology, and monitoring tools are good at showing symptoms but do not say what to bring back first.

1.2 Objective and Scope

Horizon reverses the usual pipeline. Instead of “alert, human diagnosis, ad-hoc restart”, it follows this path: continuous observation, failure detection, blast-radius computation on the current dependency graph, dynamic generation of a topological recovery sequence, risk classification of every step, automatic execution of low-risk actions, a mandatory pause before high-risk ones, health verification before any downstream step, and an immutable audit trail.

The question we examine is not whether any one of these steps is new. It is whether the closed-loop, topology-

driven generation and gated execution of the recovery sequence is a distinguishable technical contribution.

1.3 Approach and Intellectual Honesty

We have deliberately taken a sceptical stance. We inspected the real implementation (the GitHub repository *Precise-Goals/horizon* and the live deployment the team refers to), mapped each claimed feature to code, surveyed academic and commercial prior art, and read the relevant issued patents. We do not assert that the invention is patentable. We try to show where novelty might still exist and where a claim would be easy to attack.

2. Technical Field of the Invention

The invention belongs to the field of computing infrastructure management, specifically autonomous failure recovery in distributed systems. It addresses the ordered restoration of interdependent services (databases, caches, authentication services, gateways and applications) after one or more of them have failed, using a live model of how those services depend on each other.

3. Background and Review of Prior Art

3.1 Academic Self-Healing and AIOps Research

Self-healing systems and AIOps have been studied for well over a decade. Early work on checkpoint-restart and dependency-aware rollback in task-based runtimes already used explicit dependency information to cut recovery cost [1]. More recent surveys of AIOps for failure management in the era of large language models catalogue the detection, diagnosis and remediation pipelines in use and note the growing role of LLMs in proposing root-cause hypotheses [2]. Agentic architectures for incident resolution at hyperscale describe multi-agent orchestration that plans remediation with dependencies and ordering constraints in mind [3]. AUTO-OPS goes further and describes a decision layer with dependency-aware event correlation, non-compensatory admission gates, bounded remediation and post-action verification [4]. Taken together, these works show that dependency-aware planning and gated automation are active research areas, not empty ground.

3.2 Commercial Orchestration and AIOps Platforms

StackStorm offers event-driven automation and auto-remediation through a rules engine and workflow layer. It is widely used for reactive remediation, but it leans mainly on pre-authored packs rather than sequencing recovery from live topology. **Rundeck** specialises in operational job orchestration with strong audit and self-service features, yet recovery sequences are normally written as jobs or Ansible playbooks, not generated from a live dependency graph at the moment of failure.

Kubernetes controllers provide limited self-healing (restarts, rescheduling, readiness and liveness probes) at pod and deployment level. They do not keep a cross-service application dependency graph and do not build multi-tier recovery plans. **Enterprise AIOps platforms** such as Dynatrace, Datadog, Splunk and ServiceNow ITOM are strong at detection and correlation and sometimes at recommendations, but autonomous execution with dependency-constrained ordering is still partial and usually bound by policy.

Rubrik Autonomous Business Recovery (ABR) for cloud applications recovers a minimum viable business from a clean point, with explicit ordering across network, compute and data layers [5]. These products show that sequenced, policy-gated recovery has real commercial value and is already partly productised.

3.3 Patent Literature on Dependency-Mapped Recovery

The closest issued patent we found is **US12556441B1** (Bank of America, filed 2024, granted 2026), titled “System and method for computing infrastructure and operational restoral through automated dependency mapping”. Its claims cover monitoring computing devices, identifying services and infrastructure components, and generating a map of their interconnections, explicitly including graph-database representations with directional dependency edges. They also cover detecting a technical issue and dynamically generating an ordered remediation plan from that map, including ordered remediation of several issues according to a detected chain of dependencies [6].

Two further documents matter. US20230205657A1 describes automated service monitoring and remediation that evaluates dependencies before acting [7], and earlier work on automatic discovery of network software relationships supports ordered restoration [8]. Between them, these documents set a high bar for any claim that merely “builds a dependency graph and recovers in topological order”.

3.4 Prior-Art Comparison

Table 1. Selected prior art compared with Horizon

Technology / Patent	Year	Core Mechanism	Dep. Aware	Dyn. Seq.	Similarity
US12556441B1 (Bank of America)	2026	Automated dependency map + ordered remediation	Yes	Yes	5: Extremely similar
US20230205657A1	2023	Dependency-aware service remediation	Yes	Partial	4: Very high
Rubrik ABR	2026	Sequenced minimum-viable-business recovery	Yes	Yes	4: Very high

Technology / Patent	Year	Core Mechanism	Dep. Aware	Dyn. Seq.	Similarity
StackStorm / Rundeck	2010s	Event-driven / job orchestration	Limited	Static packs	3: High (execution)
Kubernetes controllers	2015+	Pod-level self-healing	Local	No cross-service	2: Moderate
AUTO-OPS (paper)	2026	Dependency-aware gates + bounded remediation	Yes	Yes	4: Very high
Horizon (this work)	2026	Live DAG + topological recovery + risk gates	Yes	Yes	n/a

Similarity scale: 1 = low, 5 = extremely similar. US12556441B1 is the single most threatening reference against any broad claim of “dependency-mapped ordered recovery”.

4. Problem Statement and Research Questions

The existing approaches above either detect without sequencing, sequence from static artefacts, or sequence from a dependency map without, as far as we can tell, the risk-gated and health-verified progression that Horizon adds. The problem we address is therefore how to restore interdependent services in the right order, at the speed of automation, without removing human control over the risky steps.

Our central research question is:

RQ1 (central): Can a system that maintains a live infrastructure dependency graph, computes blast radius on failure, dynamically generates a dependency-constrained recovery sequence, executes low-risk steps autonomously and requires explicit approval for high-risk steps measurably reduce MTTR and dependency-violation errors compared with static runbooks and unaided manual recovery?

Five supporting questions follow from it:

RQ2. How accurately does the blast-radius computation match the downstream impact observed under controlled failure injection?

RQ3. What is the end-to-end latency from failure detection to the first safe recovery action, and how does approval latency affect overall MTTR?

RQ4. How do recovery success rate and the number of dependency-order violations compare across manual recovery, a static playbook and Horizon’s dynamic sequencing?

RQ5. What failure modes appear when the dependency graph is stale, circular or incomplete, and does the system fail safely?

RQ6. Does the combination of topological sequencing, risk-classified approval gates and cryptographic audit give

a technically distinguishable control flow compared with the closest prior art?

5. Summary of the Invention

Horizon is a computer-implemented system and method for recovering failed infrastructure in dependency order. On detecting a failure, it computes the set of downstream nodes affected, derives a recovery order for that affected subgraph in which every node is restored only after the nodes it depends on are healthy, and classifies each recovery step by risk. Low-risk steps run automatically. High-risk steps suspend the workflow until a human, or a signed cryptographic transaction, approves them. Each recovered node is health-checked before its dependants are touched, and every transition is written to an audit trail.

In short, the system turns the live topology into an executable, supervised recovery plan. The plan is generated at failure time instead of being written in advance, and the person stays in control of the dangerous steps.

6. Detailed Description of the Invention

6.1 System Overview

Horizon is built as a monorepo. The frontend uses React 19, Vite and Bun and contains an in-browser recovery engine written in pure TypeScript. Optional FastAPI services exist, Firebase Realtime Database holds shared state, and an integration with the MST blockchain testnet provides cryptographic approval signatures. The core logic lives in a `Dependency Graph` class (`apps/web/src/engine/dependencyGraph.ts`, with a matching Python reference). It maintains adjacency and reverse-adjacency maps, detects cycles with a depth-first search, computes the topological recovery order with foundational nodes first, and computes the blast radius with a breadth-first search over reverse edges.

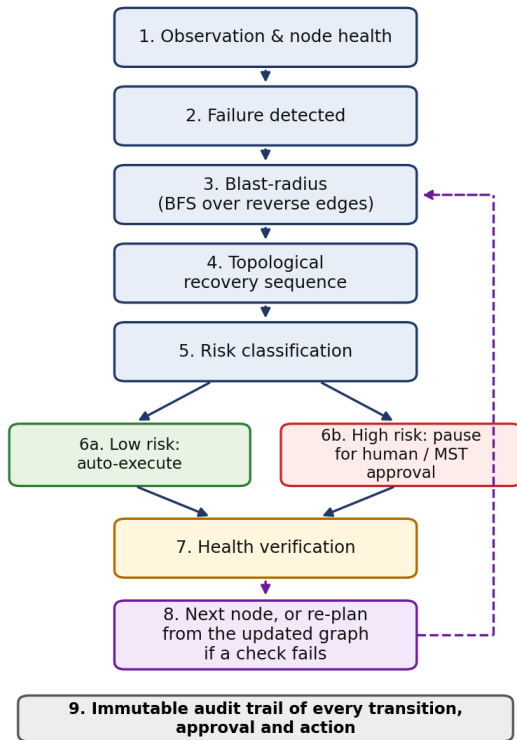


Fig. 1. Control flow of the Horizon recovery loop, from observation to audit trail, including the pause for approval and the re-plan path.

6.2 Observation and State

A ClusterStateManager creates and maintains a set of SystemNode objects (database, cache, application and gateway types), each with an explicit list of dependencies. Health status is updated by simulated probes or by external monitoring hooks. When a node's status changes to failed, the engine starts blast-radius analysis and recovery planning.

6.3 Dynamic Dependency Sequencing Engine (DDSE)

Given a failed node, the engine works through five stages:

- It computes the set of downstream affected nodes and the cascade depth.
- It derives a topological order of the affected subgraph, so that a node is recovered only once its upstream dependencies are healthy.
- It classifies every recovery step as low-risk (executed automatically) or high-risk (requires approval).
- It executes the steps one at a time and verifies health before moving to the next.
- It records every transition in an audit log.

A high-risk step pauses the workflow and raises an approval request. In the current build, approval can be given through the user interface or by a signed transaction on the MST testnet.

6.4 Implementation Status

The current prototype runs on a simulated cluster of seven nodes: a PostgreSQL primary and replica, Redis, an Auth Service, an API Gateway, a Web Frontend and a Payment Service. Recovery actions are either simulated or executed against local Docker or Kubernetes targets where these are configured.

An LLM component (a Sarvam AI integration) handles natural-language operator commands and diagnostic suggestions. The critical path, meaning graph construction, topological ordering, blast-radius computation and approval gating, stays in deterministic TypeScript, and the core recovery loop needs neither model downloads nor external LLM calls. The blockchain approval path exists but is optional for basic operation. We state all of this so that nobody reads the paper as claiming “fully autonomous production recovery across arbitrary enterprise estates”, which Horizon does not do.

7. Novelty Analysis

7.1 Decomposition of Technical Mechanisms

To judge novelty fairly we break Horizon into separate mechanisms and compare each with known practice.

Table 2. Novelty assessment by mechanism

Mech.	Description	Assessment against prior art	Novelty
A	Live, maintained dependency graph	Well known. Service maps, CMDB population, eBPF/OTel-derived graphs and commercial discovery engines are common. Horizon's version is a lightweight in-memory DAG.	Low
B	Failure-triggered blast-radius analysis	Known in concept (CMDB impact analysis, cascade modelling). Horizon's BFS over reverse edges is a standard graph algorithm.	Low
C	Dynamic recovery sequence from current topology	Partly known. US12556441B1 explicitly claims dynamic generation of an ordered remediation plan from a dependency map, and commercial ABR systems also sequence recovery. Horizon's live topological sort at failure time lines up with this art.	High risk for broad claims
D	Dependency-constrained execution with health checks between steps	Known in orchestration (Kubernetes readiness, workflow engines with success conditions). Combined with a live graph it sits close to the patent claims above.	Low to moderate
E	Risk classification with mandatory human or cryptographic approval	Known in principle (change-management gates, StackStorm and Rundeck approvals, human-in-the-loop AIOps prototypes). Signing approvals on a public testnet is an implementation choice, not a new algorithm.	Low
F	Complete recovery audit trail	A standard operational requirement that many platforms already provide.	Low
G	Closed-loop re-planning if an intermediate	The most promising candidate, but only if the system really re-plans from the updated graph mid-recovery instead of following a	Possible

Mech.	Description	Assessment against prior art	Novelty
	health check fails	frozen plan. The prototype's behaviour must be measured precisely before this is claimed.	

7.2 Three Levels of Novelty

- **Level 1, individual components.** Novelty is low.
- **Level 2, the combination.** Novelty is moderate at best, and contested by US12556441B1.
- **Level 3, the specific control logic.** Live topological generation, risk-gated sequential execution and mid-recovery health re-validation together are the only layer that may still support a narrow, claimable technical contribution, and only if concrete implementation details not disclosed in the cited patents back it up.

7.3 Competitive Positioning

Table 3. Capability comparison across approaches

Capability	Static Runbook	Monitoring	Orchestrator	AIOps Platform	Horizon
Failure detection	No	Yes	Via events	Yes	Yes (sim.)
Dependency mapping	Manual	Partial	Limited	Often	Yes (DAG)
Dynamic recovery planning	No	No	Static jobs	Partial	Yes
Dependency-aware sequencing	Author-dep.	No	Rare	Emerging	Yes (topo.)
Autonomous low-risk execution	No	No	Yes	Policy-bound	Yes
Human approval gates	Process	No	Yes	Yes	Yes
Blast-radius analysis	No	Partial	No	Often	Yes
Health validation between steps	Manual	N/A	Configurable	Variable	Yes
Audit trail	Partial	Logs	Yes	Yes	Yes
MTTR tracking	External	Yes	Partial	Yes	Yes

8. Patentability Assessment

This section is a technical assessment only. It is not legal advice.

8.1 Novelty

Broad claims covering “building a dependency graph and recovering in dependency order” collide directly with US12556441B1 and related disclosures. Narrower claims might keep a residual argument: a specific closed-loop control method that regenerates the topological sequence when an intermediate health check fails, a dual-path

execution model (automatic versus cryptographic approval based on risk), and a state machine that preserves recovery context across pause and resume. Even these depend on a full file-wrapper and non-patent literature review.

8.2 Inventive Step

A person skilled in SRE automation would probably see the combination of known graph algorithms, known approval gates and known audit logging as an obvious engineering aggregation. That view changes only if a surprising technical effect is demonstrated, for example a measured reduction in dependency-violation errors that the closest prior art cannot achieve.

8.3 Industrial Applicability, Enablement and Eligibility

Industrial applicability is clear, because the system addresses a recognised operational problem. On **enablement**, the public GitHub repository and the deterministic TypeScript engine give a concrete, enabling disclosure of the core graph and sequencing logic. On **subject-matter eligibility**, pure software methods that improve the operation of concrete computing infrastructure, such as the ordered restoration of failed services, are more likely to be treated as technical than as pure business methods, but the answer depends on jurisdiction and needs specialist advice. In India, software-related inventions are examined for technical effect or technical contribution, so the invention should be framed as an improvement in the reliability and recovery behaviour of real distributed computing systems.

9. Security Considerations

Autonomous recovery enlarges the attack surface. The main risks we identified are:

- compromised automation credentials running destructive playbooks;
- dependency-graph poisoning that inverts the recovery order;
- approval bypass or replay of approval tokens;
- tampering with the audit log;
- recovery-loop attacks that keep restarting healthy services;
- monitoring spoofing that fabricates failures; and
- ransomware scenarios in which the recovery system itself becomes the vector.

The controls that should be in place, and that the prototype realises only in part, are least-privilege execution identities, signed and versioned playbooks, immutable or append-only audit storage, multi-party or threshold approval for the highest-risk classes, graph integrity

checks, and circuit-breakers that halt automation after repeated failures or unexpected growth in blast radius. The blockchain approval path gives non-repudiation of human decisions, but on its own it does not protect the execution plane.

10. Failure-Safe Design and Limitations

The system has to fail safely when the dependency graph is wrong or stale, when health checks give false positives, when a recovery action itself fails, when approval never arrives, when the executor is unavailable, when circular dependencies exist, and when several nodes fail at once.

Current mitigations are cycle detection (hasCycle), severity labelling of the blast radius, and explicit gating of high-risk steps. Some gaps remain: a partially applied recovery sequence cannot be rolled back automatically, the generated plan has no formal model-checking, and the simulated environment does not yet stress multi-region or multi-cluster topologies. We list these as limitations, not features.

11. Experimental Status and Evaluation Plan

11.1 Current Status

Project documentation mentions preliminary observations of a substantial MTTR reduction and of dependency hierarchy being preserved in simulation. We treat these as prototype observations, not as statistically validated results. The sample size, variance, baseline definition and number of independent runs have not been published in a form that supports strong quantitative claims.

11.2 Proposed Evaluation

A rigorous evaluation would compare four baselines: (1) unaided manual recovery by an operator who has only monitoring dashboards; (2) a static runbook or fixed playbook; (3) Horizon with dependency sequencing switched off (random or alphabetical order); and (4) the full Horizon pipeline.

It would measure MTTR, mean time to detect (MTTD), recovery success rate, the number of dependency-order violations, the number of operator interventions, approval latency, blast-radius precision and recall against a ground-truth cascade, and the number of failed recovery actions.

The test scenarios should include a single database failure, an API failure, a cache failure, a simultaneous multi-node failure, a stale graph, injected circular dependencies, an approval timeout, and a recovery action that fails part-way through a sequence.

12. Potential Claim Concepts (Non-Legal)

The concepts below are research notes only. Final claim language must be drafted by qualified patent counsel.

Independent claim concept (method). A computer-implemented method comprising: maintaining a directed dependency graph of infrastructure nodes; detecting a failure of at least one node; computing a blast-radius set of downstream nodes; generating a topological recovery sequence constrained by the graph; classifying each recovery action by risk; automatically executing low-risk actions; suspending execution and obtaining cryptographic or human approval before high-risk actions; verifying the health of a recovered node before allowing a dependent node to recover; and recording an immutable audit trail of the sequence.

Dependent claim concepts:

- weighted edges in the dependency graph;
- dynamic graph updates from live telemetry;
- mid-recovery re-planning when a health check fails;
- action adapters for multiple environments (container, VM, bare metal);
- approval timeout with safe abort; and
- rollback of a partially applied sequence.

13. IPR Roadmap (Indian Context)

Table 4. Staged route to a provisional filing

Phase	Step	What to do
1	Freeze the invention disclosure	Lock the architecture description, the code commit hash and the experimental protocol. Record each inventor's contribution (Sarthak Tulsidas Patil, Karan Verma, Sneha Sharma, Sneha Patidar).
2	Formal prior-art search	Commission or carry out an expanded search across USPTO, EPO, WIPO and Indian Patent Office records, paying special attention to the Bank of America family and AIOps remediation patents.
3	Patentability opinion	Obtain a written counsel opinion on whether residual novelty exists for the narrow closed-loop control claims.
4	Provisional application (India)	If counsel supports residual novelty, file a provisional specification stressing live topological re-planning and the risk-gated state machine, backed by the enabling code and any measured technical effect.
5	Prototype evidence	Collect controlled experimental data under the plan in Section 11 and attach the results to the complete specification.
6	Complete specification and/or PCT	Within the priority year, file the complete specification and, if international protection is wanted, a PCT application designating the relevant states.

Counsel who know Indian Patent Office practice on computer-related inventions should be retained throughout.

14. Conclusion and Verdict

Horizon is a coherent, working prototype of dependency-aware, gated autonomous recovery on a simulated multi-

tier stack. The engineering is real: the DAG, the topological order, the blast-radius BFS, the risk gates and the audit log all exist in code and can be demonstrated. Against the strongest prior art, though, the research contribution is narrower than a first reading of the product description might suggest. The combination of live topological sequencing, mandatory approval for high-risk steps and health-verified progression is still the most defensible technical ground, and even that is contested by existing patents and commercial systems.

What is genuinely strong. A clean, deterministic core engine that makes the recovery order explicit and inspectable; an honest separation of deterministic control from optional LLM assistance; and a deployable demonstration that explains the problem clearly.

What is already known. Dependency graphs for recovery ordering, blast-radius analysis, automated remediation with approval gates, and sequenced disaster recovery.

What should not be claimed as novel. The mere existence of a dependency graph, the mere use of topological sort, or “AI-powered recovery” with no specific, non-obvious control mechanism behind it.

What to strengthen before any filing. (1) A measured technical effect against the baselines in Section 11; (2) explicit mid-recovery re-planning behaviour; (3) a formal treatment of stale-graph and circular-dependency failure modes; and (4) counsel-reviewed claim scope that steers around US12556441B1.

For the hackathon. Lead with the live demonstration of correct recovery order, the visible approval gate, the blast-radius visualisation and the audit timeline. These are more persuasive than broad claims about patentability.

Table 5. Overall assessment

Dimension	Rating
Overall technical differentiation	6 / 10
Novelty potential (narrow claims)	4 to 5 / 10
Patentability risk	High (broad claims); Medium (narrow closed-loop claims)
Implementation readiness (prototype)	7 / 10
Enterprise potential (with further hardening)	7 / 10
Hackathon impact (demo and clarity of problem)	8 / 10

Patent counsel must review the contents of this paper before any filing decision is made. Nothing here is legal advice.

References

- [1] K. Dichev et al., “Dependency-Aware Rollback and Checkpoint-Restart for Distributed Task-Based Runtimes,” arXiv:1705.10208, 2017.
- [2] L. Zhang et al., “A Survey of AIOps for Failure Management in the Era of Large Language Models,” arXiv:2406.11213, 2024.

- [3] “Autonomous Incident Resolution at Hyperscale: An Agentic AI Architecture for Network Operations,” arXiv preprint, 2026.
- [4] S. K. Vytla, “AUTO-OPS: Predictive Incident Detection and Self-Healing Remediation for Cloud-Native Enterprise Platforms,” Engineering, Technology & Applied Science Research, 2026.
- [5] Rubrik, “Autonomous Business Recovery,” product and technical descriptions, 2026.
- [6] J. H. Kling et al., “System and method for computing infrastructure and operational restoral through automated dependency mapping,” U.S. Patent US12556441B1, Bank of America Corp., 2026.
- [7] “Techniques for automated service monitoring and remediation in a distributed computing system,” U.S. Patent Application US20230205657A1, 2023.
- [8] “Systems and methods for automatic discovery of network software relationships,” U.S. Patent Application US20100223609A1, 2009.
- [9] Horizon project source code and architecture documentation, <https://github.com/Precise-Goals/horizon>, inspected October 2026.
- [10] National and commercial AIOps, StackStorm, Rundeck and Kubernetes documentation (public vendor and project sources), 2015–2026.